

Leaving the lead glas open:

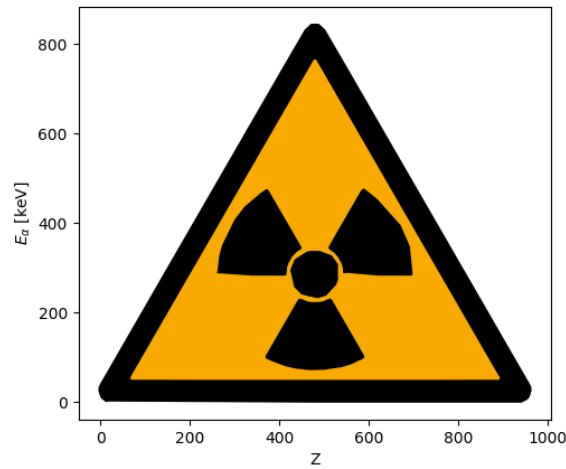


Figure 0.1: Attention, Attention! Dear comrades! The lab council reports that [...] an unfavorable radiation situation is developing in the lab rooms. [...] AGAIN?^{5 6}

Analysis of experiment 255 - X-Ray Spectroscopy

“Physikalisches Praktikum 2.2”

in the physics department of
university Heidelberg

by

Benjamin Kleijn

August 11, 2026

lab date 30.6.2026

group 9

tutor Friso Grace

Contents

1. Introduction	4
1.1. Motivation	4
1.2. Experiment goals	4
1.3. Setup and devices	4
1.4. Theoretical background	6
1.5. X-ray Radiation	6
1.6. Bragg Reflection	7
1.7. Crystals and Lattice Structure	7
2. Procedure and data protocol	9
2.1. Measurement methods	9
2.2. Lab protocol / data sheet	9
3. Data analysis	12
3.1. Plancks constant via critical wavelength	14
3.2. $K_{\alpha,\beta}$ lines with LiF	15
3.3. Plancks constant via starting voltage	17
3.4. $K_{\alpha,\beta}$ lines with NaCl	18
4. Discussion	20
4.1. spectrum	20
4.2. critical wavelength and critical angle	20
4.3. Significance of h results	20
4.4. $K_{\alpha,\beta}$ lines with LiF	20
4.5. starting voltage	21
4.6. NaCl crystal examination	21
4.7. Avogadro constant	22
4.8. conclusion	22
Literature references	23
Figure references	23
A. Python-code	23

List of Figures

0.1. <i>Meme</i> : chernobyl evacuation announcement	1
1.1. experiment setup ¹	5
1.2. Bragg Scattering ¹	7
3.1. X-ray spectrum of Mo with LiF-crystal	14
3.2. $K_{\alpha,\beta}$ Gaussian fits for LiF	16
3.3. operating voltage	17
3.4. X-ray spectrum of Mo with NaCl-crystal	18
3.5. $K_{\alpha,\beta}$ Gaussian fits for NaCl	19

List of Tables

4.1. comparison of $h_{\vartheta,U}$ and h_{lit} ²	20
4.2. comparison of $K^{\text{exp}}(\text{Mo})$ and $K^{\text{Lit}}(\text{Mo})$ ¹	20
4.3. comparison of a_{exp} and a_{lit} ³	21
4.4. comparison of N_A^{exp} and N_A^{lit} ⁴	22

1. Introduction

1.1. Motivation

X-ray radiation is one important type of radiation and poses an interesting subject for physics, thus its mechanics need to be understood in order to provide the necessary precision for medical purposes and other applications. The following experiment introduces X-ray radiation, its generation and analysis, and provides the basics of X-Ray spectroscopy, which are used for experiment 256: *X-ray fluorescence*¹. In this experiment however, the aim is, on one hand, to determine fundamental constants of physics experimentally and, on the other hand, to analyze the structure of crystals using diffraction methods. Specifically, Planck's quantum of action h is to be determined from the short-wavelength end of the bremsstrahlung spectrum as well as from the threshold voltage of a Lithium Fluoride (LiF) crystal. Furthermore, the wavelengths of the characteristic K_α and K_β lines of a molybdenum anode are to be determined. By using LiF and sodium chloride (NaCl) as diffraction crystals, the connection between crystal lattice structure and macroscopic quantities such as the Avogadro number N_A can also be established.

1.2. Experiment goals

This experiment is done to study X-ray mechanics: Specifically the spectrum of crystals probed with X-Rays. By using Bragg-Refraction (working with refraction angles), measuring the spectrum and accelerating voltages, one can

- determine the value of Planck's constant by two different ways
- calculate the lattice constant of a sodium chloride (NaCl) crystal, and Avogadro's number
- find the K_α and K_β peaks of molybdenum

1.3. Setup and devices

The X-ray chamber consists of a X-ray tube (a heated electrode to create fast electrons which are accelerated towards the molybdenum anode by an electric field, after hitting it, they are slowed down fastly by the anodes material emitting X-ray radiation as Bremsstrahlung). The produced X-ray spectrum is focused onto the crystal and in contrast to experiment 256, only refracted, it doesn't cause secondary X-Ray radiation. The refracted X-ray radiation is analyzed by a counter tube mounted on a rotatable decive, called goniometer, which can be moved to different angles thus allowing to measure different angles on the crystal. The results, hit counts and spectrum data are displayed on the decive and/or by a measurement programm.

1. X-ray chamber with X-ray tube and Mo-anode
2. LiF and NaCl crystal plates

3. counter tube mounted on a goniometer, like shown in fig. 1.2
4. desktop PC with measurement programm



Figure 1.1: experiment setup¹

1.4. Theoretical background¹

1.5. X-ray Radiation

Generation of X-Rays: In this experiment, the generation is achieved via an X-ray tube with a molybdenum anode and a heated filament cathode. Inside an evacuated glass vessel, electrons are accelerated by an electric field between the cathode and the anode. The kinetic energy E of the electrons results from the applied accelerating voltage U :

$$E = eU \quad (1.1)$$

where e represents the elementary charge. Upon impact with the anode, the electrons are decelerated in the Coulomb field of the atomic nuclei. This braking process leads to the emission of a continuous spectrum of electromagnetic radiation, known as the bremsstrahlung spectrum.

The maximum energy of an emitted photon is reached when an electron converts its entire kinetic energy into a single photon. This results in a minimum possible wavelength λ_{cr} , the critical wavelength (or short-wavelength limit), which is described by conservation of energy:

$$eU = h\nu_{cr} = \frac{hc}{\lambda_{cr}} \quad (1.2)$$

Here, h is Planck's quantum of action and c is the speed of light.

X-ray fluorescence: In addition to the continuous spectrum, a characteristic line spectrum is generated if the voltage is sufficiently high: High energy electromagnetic radiation can eject an atom's inner (strongly bound) electrons, analogous to the photoeffect. The resulting holes are filled by electrons of higher shells/states with larger quantum number, emitting a photon of the corresponding energy difference in the process. An electron of the second shell (K_α radiation) is most likely to fill the ejected electron's place, less likely is an electron of the third shell (K_β radiation). The terms originate by using the final shell letter, K , and the difference between initial and final shell, $\alpha \doteq 1$, $\beta \doteq 2$. For a general transition from the n -th to the m -th shell, the energy $E_{n \rightarrow m}$ can be estimated according to Moseley's Law: Starting at the Rydberg-formula (solution to the Schrödingers equation for Hydrogen), one needs to take account for effects to describe the transition energy $E_{n \rightarrow m}$ for heavier objects correctly, thus arriving at:

$$E_{n \rightarrow m} = hcR_\infty(Z - A)^2 \left(\frac{1}{m^2} - \frac{1}{n^2} \right) \quad (1.3)$$

Here, R_∞ denotes the Rydberg constant, Z the atomic number of the anode material (here molybdenum), and A a shielding constant, which is approximately $A \approx 1$ for K_α radiation ($n = 2 \rightarrow m = 1$).

1.6. Bragg Reflection

To measure the wavelengths of the generated X-ray radiation, crystals are used as diffraction gratings. Since the wavelengths of X-ray radiation are of the same order of magnitude as the interatomic distances in crystals, constructive interference occurs at the lattice planes of the crystal. This phenomenon is called Bragg reflection. An incident ray is reflected at various parallel lattice planes. For these reflected rays to overlap constructively, the path difference Δs must be an integer multiple of the wavelength λ . Bragg's Law describes this relationship:

$$2d \sin(\vartheta) = n\lambda, \quad n \in \mathbb{N} \quad (1.4)$$

Here, d is the distance between adjacent lattice planes, ϑ is the angle of incidence (glancing angle), and n is the order of reflection. By varying the angle ϑ and measuring the intensity, the X-ray spectrum can be recorded.

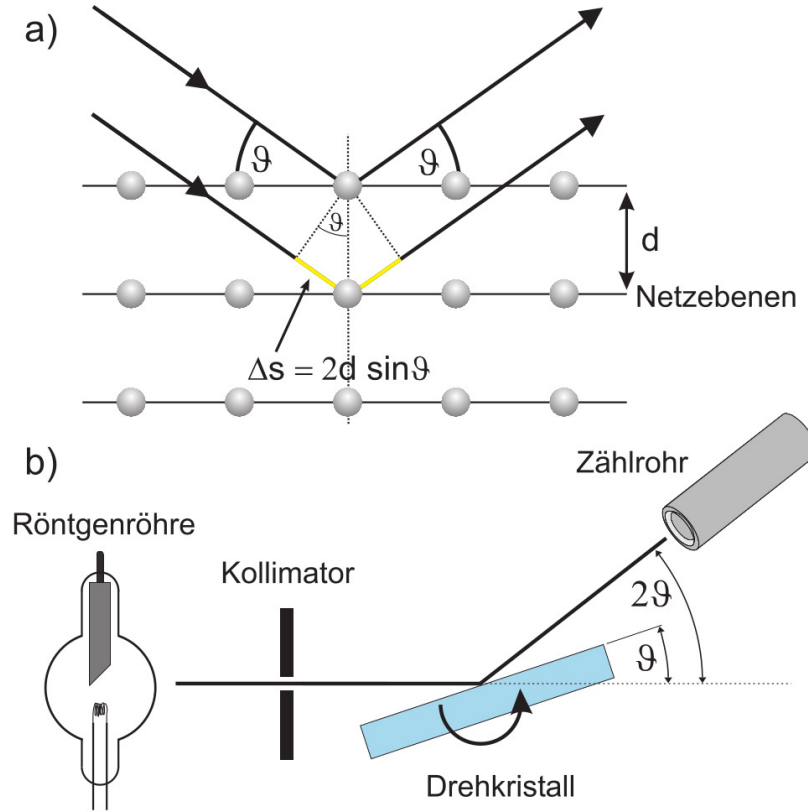


Figure 1.2: Bragg Scattering Herleitung¹

1.7. Crystals and Lattice Structure

The LiF and NaCl crystals used in the experiment both possess a face-centered cubic structure. In this structure, a unit cell repeats periodically in space. There are four atoms (or ion pairs)

per unit cell. For a cut along the principal planes, the lattice plane spacing d in terms of the lattice constant a is given by:

$$d = \frac{a}{2} \quad (1.5)$$

If the wavelength λ is known, the lattice constant a can be determined from the measured angle ϑ . Conversely, a known lattice constant enables the determination of microscopic quantities from macroscopically measurable properties. The Avogadro number N_A , which indicates the number of particles per mole, can be calculated via the volume of the unit cell. The volume of a unit cell is $V = (2d)^3$. With the molar mass M_{mol} and the density ρ of the crystal, it follows that:

$$N_A = \frac{4V_{mol}}{(2d)^3} = \frac{4M_{mol}}{\rho(2d)^3} = \frac{1}{2} \frac{M_{mol}}{\rho d^3} \quad (1.6)$$

This relationship allows the Avogadro number to be determined indirectly via X-ray diffraction.

In the evaluation, the statistical nature of radioactive or count-rate-like events is also taken into account. Since the count rate N follows a Poisson process, the standard error of a single measurement is:

$$\sigma = \sqrt{N} \quad (1.7)$$

This value is used to incorporate the uncertainty of the measured intensities into the error propagation.

2. Procedure and data protocol

2.1. Measurement methods

Task 1: The LiF-crystal is examined with the radiation set to an accelerating voltage of $U_a = 35$ kV by automatically turning the crystal from $(3 \text{ to } 22)^\circ$ in steps of $\Delta\vartheta = 0.2^\circ$ and measuring the resulting intensity of the reflected/scattered radiation with an exposure time of $t = 5$ s.

Task 2: With the plot given by the first task, one can determine the K_α and K_β lines of first and second order, and examine them a bit more in detail by only choosing the individual angle-range of said lines and scanning them with $\Delta\vartheta = 0.1^\circ$ and $t = 20$ s.

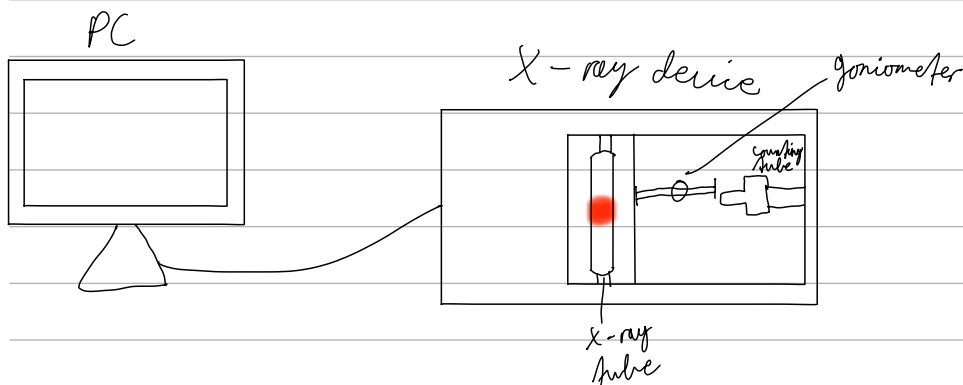
Task 3: The count rate/intensity is recorded with constant $\vartheta = 7.5^\circ$ as a function of U_a , therefore one varies U_a between $(20 \text{ to } 35)$ kV in steps of 1 kV, each time with $t = 20$ s.

Task 4: Same procedure as in task 1 is performed, this time with the NaCl-crystal, the only difference is an angle-range of $(3 \text{ to } 18)^\circ$.

2.2. Lab protocol / data sheet

Equipment

- X-ray device with X-ray tube (Mo anode)
- Goniometer
- counting tube
- LiF- and NaCl crystal
- computer
- fluorescent screen with CCD-camera

SetupTask 1: X-ray spectrum of the LiF-crystal

We aim to measure the X-ray spectrum of our Mo anode with the LiF crystal. We set the device to $U = (35 \pm 0.1) \text{ kV}$, $I = (10 \pm 0.01) \text{ mA}$, $t = 5 \text{ s}$ and $\Delta B = 0.2^\circ$. We measure values between 3° and 23° . We store our data digitally.

Task 2: K_{α} and K_{β} lines of Mo

From the previous measurement, we select the locations of the K_{α} and K_{β} lines:

1. order: K_{α} at 10.2° , K_{β} at 9.0°
2. order: K_{α} at 20.6° , K_{β} at 18.2°

We repeat the measurements of the first task with $\Delta B = 0.1^\circ$ and $t = 20 \text{ s}$.

Task 3: counting rate as a function of V

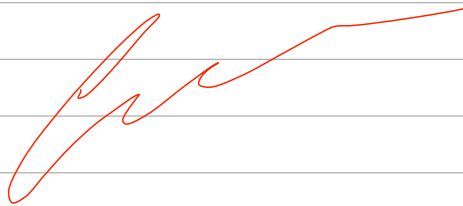
We use an angle of $\beta = 7.5^\circ$ and measuring time of $t = 20\text{s}$. We measure the counting rate for values from 20 kV to 35 kV in steps of 1 kV

Table 1: $r(V)$

Voltage V [kV]	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35
Rate r [$\frac{1}{s}$] (average over 20s)	3	9,5	6,75	10,5	41,9	122,0	183,9	237,0	296,9	338,7	393,6	440,5	483,4	521,4	573,1	609,1

Task 4: Measurement of the x-ray spectrum with a NaCl crystal

We swap out the LiF crystal for a NaCl crystal, and measure the x-ray spectrum from 3° to 18° with the same options as Task 1. We again save our data digitally.



3. Data analysis

Equations for statistical analysis¹

Varianzen: Für die statistische Auswertung von n Messwerten $\{x_i\}_{i=1}^n$ werden folgende Größen definiert:

$$\text{Arithmetisches Mittel} \quad \bar{x} = \text{Mean}(\{x_i\}_{i=1}^n) = \frac{1}{n} \sum_{i=1}^n x_i \quad (\text{S.1})$$

$$\text{Varianz} \quad \sigma^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2 \quad (\text{S.2})$$

$$\text{Standardabweichung} \quad \sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2} \quad (\text{S.3})$$

$$\text{Fehler des Mittelwerts} \quad \Delta \bar{x} = \frac{\sigma}{\sqrt{n-1}} \quad (\text{S.4})$$

Fehlerfortpflanzung: Der Fehler einer Funktion $f(x_1, \dots, x_N)$, die von den Variablen $\{x_i\}_{i=1}^N$ abhängt, wird wie folgt bestimmt:

$$\text{Gauß'sche Fehlerfortpfl.} \quad \Delta f = \sqrt{\sum_{i=1}^N \left(\frac{\partial f}{\partial x_i} \Delta x_i \right)^2} \quad (\text{S.5})$$

$$\text{relativer Fehler von } f = \prod_{i=1}^N x_i^{\alpha_i} \quad \frac{\Delta f}{|f|} = \sqrt{\sum_{i=1}^N \left(\frac{\alpha_i \Delta x_i}{x_i} \right)^2} \quad (\text{S.6})$$

Ergebnissignifikanz: Resultate eines Experiments bzw. Berechnung a_{exp} und die entsprechenden Literaturwerte a_{lit} werden folgend verglichen:

$$\text{Absolute Abweichung} \quad A = |a_{\text{lit}} - a_{\text{exp}}|$$

$$\Delta A = \sqrt{(\Delta a_{\text{lit}})^2 + (\Delta a_{\text{exp}})^2} \quad (\text{S.7})$$

$$\text{Relative Abweichung} \quad R[\%] = \frac{A}{a_{\text{lit}}} \cdot 100 \quad (\text{S.8})$$

$$\text{Signifikanz} \quad S[\sigma] = \frac{A}{\Delta A} = \frac{|a_{\text{lit}} - a_{\text{exp}}|}{\sqrt{\Delta a_{\text{lit}}^2 + \Delta a_{\text{exp}}^2}} \quad (\text{S.9})$$

Fitgüte: Für einen Fit mit Funktionswerten $\{F_{a_1, \dots, a_m}(x_i)\}_{i=1}^n$ und Fitparametern $\{a_j\}_{j=1}^m$ an die experimentellen Messwerte $\{y_i\}_{i=1}^n$ mit Fehler $\{\Delta y_i\}_{i=1}^n$ kann die Güte des Fits mit diesen Größen analysiert werden:

$$\chi^2 \text{ Summe} \quad \chi^2 = \sum_{i=1}^n \left(\frac{F(x_i) - y_i}{\Delta y_i} \right)^2 \quad (\text{S.10})$$

$$\text{Freiheitsgrade} \quad f = n - m \quad (\text{S.11})$$

$$\text{normierte reduzierte } \chi^2 \text{ Summe} \quad \chi_{\text{red}}^2 = \frac{\chi^2}{f} \quad (\text{S.12})$$

$$\text{Fitwahrscheinlichkeit} \quad p = 1 - \text{chi2.cdf}(\chi^2, f) \quad (\text{S.13})$$

Hierbei wird das Modul `chi2` von `scipy.stats` benutzt.

3.1. Plancks constant via critical wavelength

After extrapolating the linear trend of the first values of the bremsstrahlungs-spectrum, one can determine the critical angle by using the intersection of the linear fit with the spectrum's underground noise ($I = 100$ a.u.). Its error is derived by plotting all possible linear fits (dotted lines $g_{m',c'}(\vartheta)$ in fig. 3.1b) with the parameters m', c' varying according to their errors in the **pcov**-matrix ($m' = m \pm \Delta m$), and then one can choose the maximal possible derivation $\max |\frac{100-c}{m} - \frac{100-c'}{m'}|$ as $\Delta\vartheta$. This angle $\vartheta = (4.9 \pm 1.0)^\circ$ corresponds to the critical wavelength via eq. (1.4):

$$\lambda_{cr} \approx \frac{2d\vartheta}{n} \quad \Delta\lambda_{cr} = 2\frac{\Delta\vartheta d}{n} \quad \boxed{\lambda_{cr} = (35 \pm 8) \text{ pm}} \quad (3.1)$$

We used the small angle approximation and $d = 201.4 \text{ pm}^1$. By using eq. (1.2), $U_a = (35.00 \pm 0.05) \text{ kV}$ and the natural constants speed of light c and elementary charge e , we can calculate a value for h :

$$h = \frac{U_a e \lambda_{cr}}{c} \quad \Delta h = h \sqrt{\frac{\Delta U_a^2}{U_a^2} + \frac{\Delta \lambda_{cr}^2}{\lambda_{cr}^2}} \quad \boxed{h = (6.5 \pm 1.5) \cdot 10^{-34} \text{ J s}} \quad (3.2)$$

second order spectrum: the second order spectrum ($n = 2$) begins at:

$$\vartheta^{2\text{nd}} = \frac{n\lambda_{cr}}{2d} \quad \Delta\vartheta^{2\text{nd}} = \frac{\Delta\lambda_{cr} n}{2d} \quad \boxed{\vartheta^{2\text{nd}} = (9.7 \pm 2.3)^\circ} \quad (3.3)$$

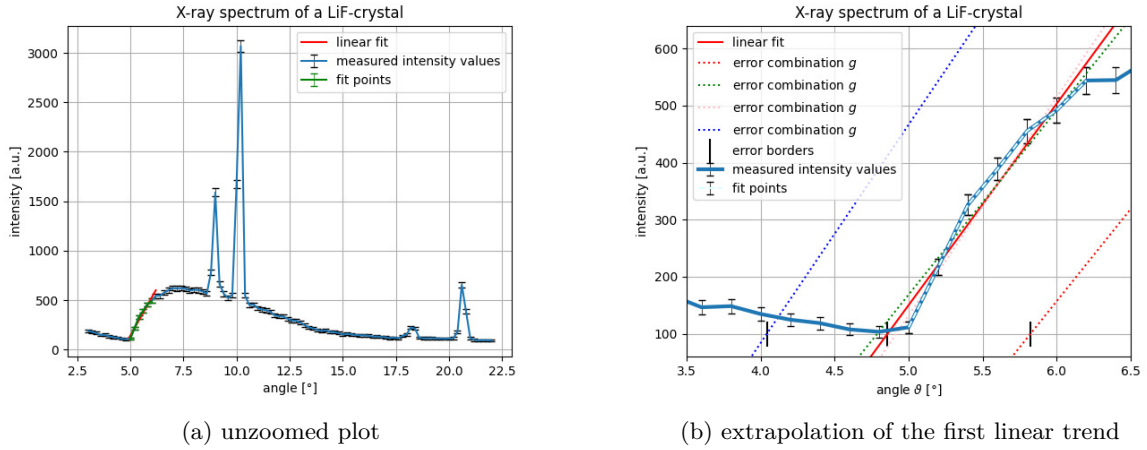


Figure 3.1: X-ray spectrum of Mo with LiF-crystal

3.2. $K_{\alpha,\beta}$ lines with LiF

Mean μ and the full half width FWHM of gaussian curves fitted on the primary and secondary spectral peaks in fig. 3.2 are used for determining the angles/wavelengths K_α , K_β and their respective errors. This is done by using the following gaussian function:

$$f(\vartheta) = \frac{A}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{\vartheta-\mu}{\sigma}\right)^2} + c \quad (3.4)$$

Where A is the area under the curve and c the level of the underground noise. One needs to provide starting values for `curvefit`, this can be done by using the measured intensity and angle values.

$$c = \text{np.min(intensity)} \quad A = \overbrace{(\text{np.max(intensity)} - \text{np.min(intensity)})}^{f(\mu)} \sigma\sqrt{2\pi} \quad (3.5)$$

the last values are estimated: $\sigma = 0.1^\circ$ and for μ , we look at the graphs and estimate by hand/eyes (one could have also used $\min(\vartheta) + \frac{\max(\vartheta) - \min(\vartheta)}{2}$).

A neat little thing done in fig. 3.2 is to add a second x-axis for displaying the wavelength λ : It corresponds to the angle ϑ according to eq. (1.4):

$$\lambda = \frac{2d \sin(\vartheta)}{n} \quad \text{FWHM[m]} = \Delta\lambda = \frac{2d \cos(\overbrace{\vartheta}^{\mu[\text{rad}]}) \overbrace{\Delta\vartheta}^{\text{FWHM}[\text{rad}]}}{n} \quad (3.6)$$

The error formula is used to transform the FWHM measured in radians [rad] to [m], FWHM is set as the error of $\mu = \lambda$, that's why $\text{FWHM} = \Delta\lambda$. To calculate FWHM on the firsthand, we can use the condition $\frac{1}{2}(f(\vartheta = \mu) - c) = f(\vartheta = \mu + \vartheta_{\frac{1}{2}}) - c$ to get $\text{FWHM} = 2\vartheta_{\frac{1}{2}} = 2\sigma\sqrt{2\ln(2)}$. σ and μ are provided by the fit results/parameters of `curve_fit`.

We arrive at the following values for the $K_{\alpha,\beta}^n$ -lines:

$K_\alpha^1 = (71.0 \pm 1.7) \text{ pm}$	$K_\alpha^2 = (71.1 \pm 1.0) \text{ pm}$	$S[\sigma] = 0.06$
$K_\beta^1 = (63.0 \pm 1.7) \text{ pm}$	$K_\beta^2 = (63.2 \pm 1.0) \text{ pm}$	$S[\sigma] = 0.11$

The errors represent the FWHM of each peak, Specifically $\text{FWHM}_\alpha^1 = (1.67 \pm 0.03) \text{ pm}$. We calculated this by using eq. (3.6) with $\vartheta = \mu$, $\sigma[^\circ]$ and taking $\Delta\sigma[^\circ] = \sqrt{\text{pcov}[2][2]}$ out of the `curve_fit` covariance matrix. Firstly, we change μ, σ to [rad], then:

$$\text{FWHM}[\text{rad}] = 2\sigma\sqrt{2\ln(2)} \quad \Delta\text{FWHM}[\text{rad}] = 2\Delta\sigma\sqrt{2\ln(2)} \quad (3.7)$$

$$\text{FWHM}[\text{m}] = 2d \cos(\mu) \text{FWHM}[\text{rad}] \quad \Delta\text{FWHM}[\text{m}] = 2d \cos(\mu) \Delta\text{FWHM}[\text{rad}] \quad (3.8)$$

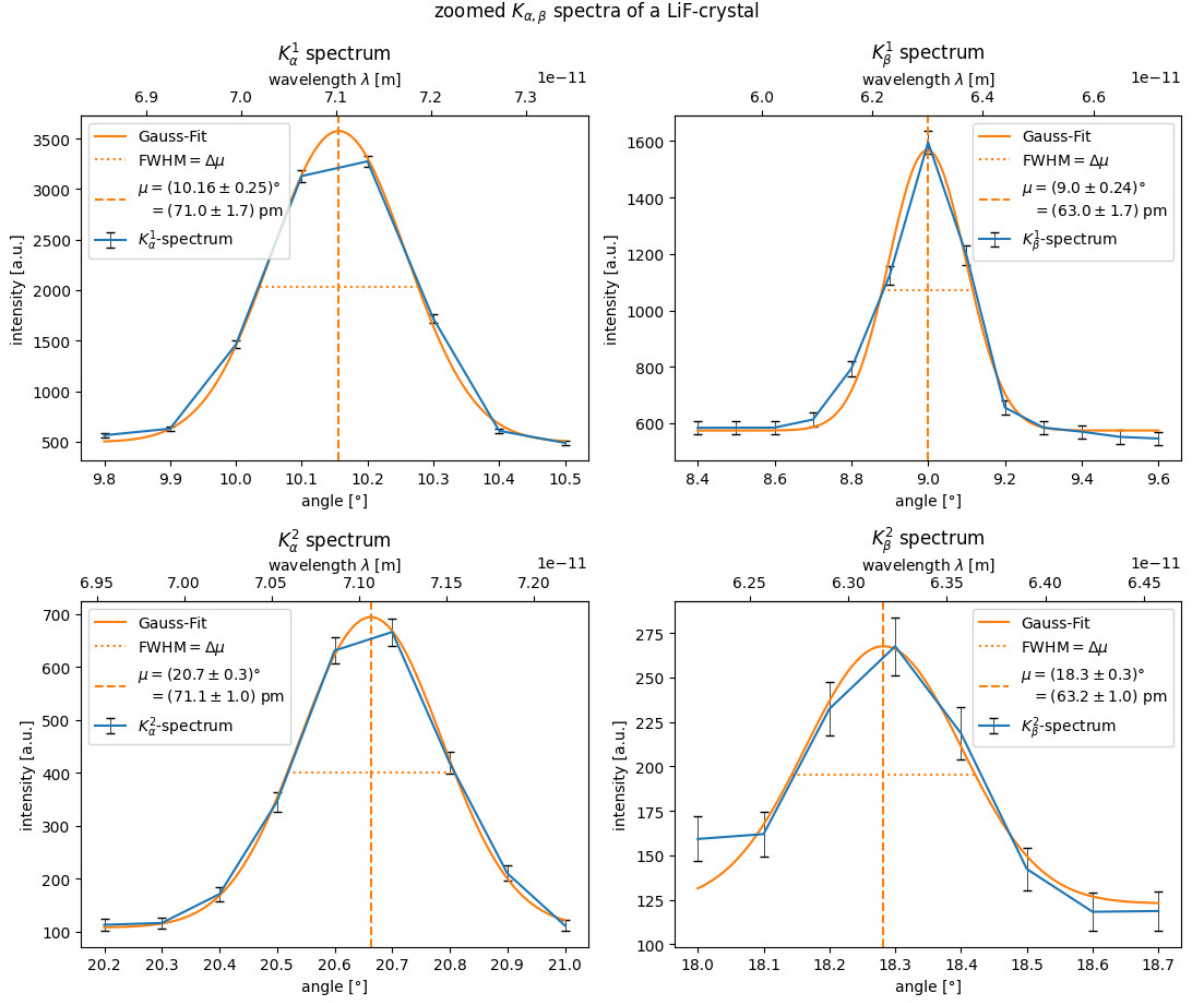


Figure 3.2: $K_{\alpha,\beta}$ Gaussian fits for LiF

Interestingly, one can arrive at FWHM [m] in a different way:

$$\text{FWHM[m]} = \mu + \frac{\overbrace{\text{FWHM[rad]}}^{\vartheta}}{2} - \left(\mu - \frac{\overbrace{\text{FWHM[rad]}}^{\vartheta}}{2} \right) = \frac{2d}{n} [\sin(\mu + \vartheta) - \sin(\mu - \vartheta)] \quad (3.9)$$

$$= \frac{2d}{n} 2 \cos(\mu) \sin(\vartheta) \quad \vartheta \ll 1 \approx \frac{2d}{n} \cos(\mu) \text{FWHM[rad]} \quad (3.10)$$

I think, its a coincidence, that eq. (3.6) is the same formula as the one above, eq. (3.10) seems way more intuitive. In eq. (3.6), we assumed, that $\Delta\lambda = \Delta\mu = \text{FWHM}$, and that is a gross assumption rooted in overestimating errors.

3.3. Plancks constant via starting voltage

The angle ϑ was set to 7.5° , now the intensity can be plotted against the operating voltage to extrapolate the minimal voltage U_a required to release electrons with $\lambda(\vartheta)$. We determine: $U_a = (22.3 \pm 1.8) \text{ kV}$ by using $U_a = \frac{-c}{m} = \{U | f(U) = 0\}$. ΔU_a is determined by plotting all possible linear fits (dotted lines $g_{m',c'}(U)$ in fig. 3.3) with the parameters m', c' varying according to their errors in the `pcov`-matrix, and then one can choose the maximal possible derivation $\max |\frac{-c}{m} - \frac{-c'}{m'}|$ as ΔU_a .

Once again, we use eq. (3.1) and eq. (3.2), we can calculate the values for λ and followingly h :

$$\lambda_{cr} = (52.7 \pm 0.8) \text{ pm} \quad h = (6.3 \pm 0.6) \cdot 10^{-34} \text{ J s}$$

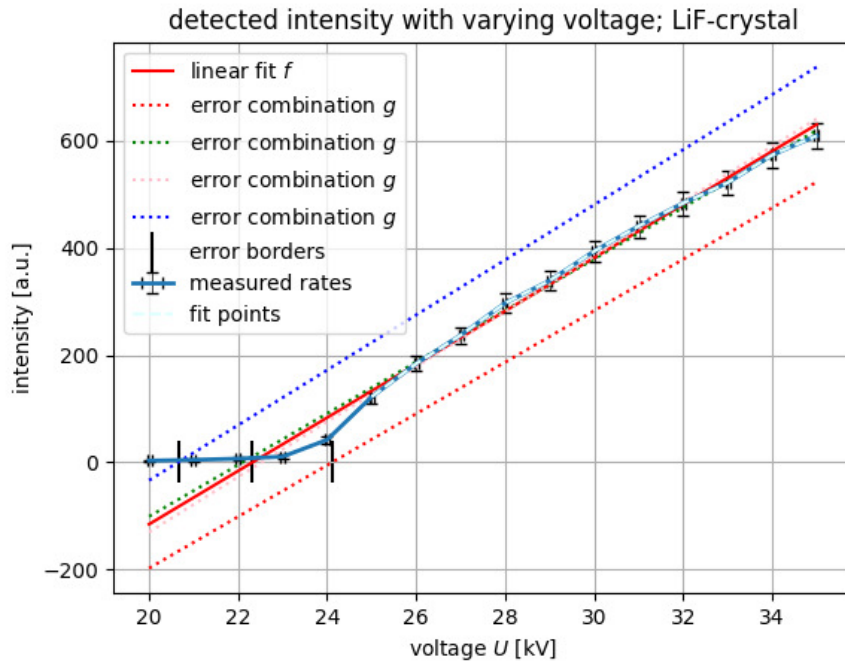


Figure 3.3: operating voltage

3.4. $K_{\alpha,\beta}$ lines with NaCl

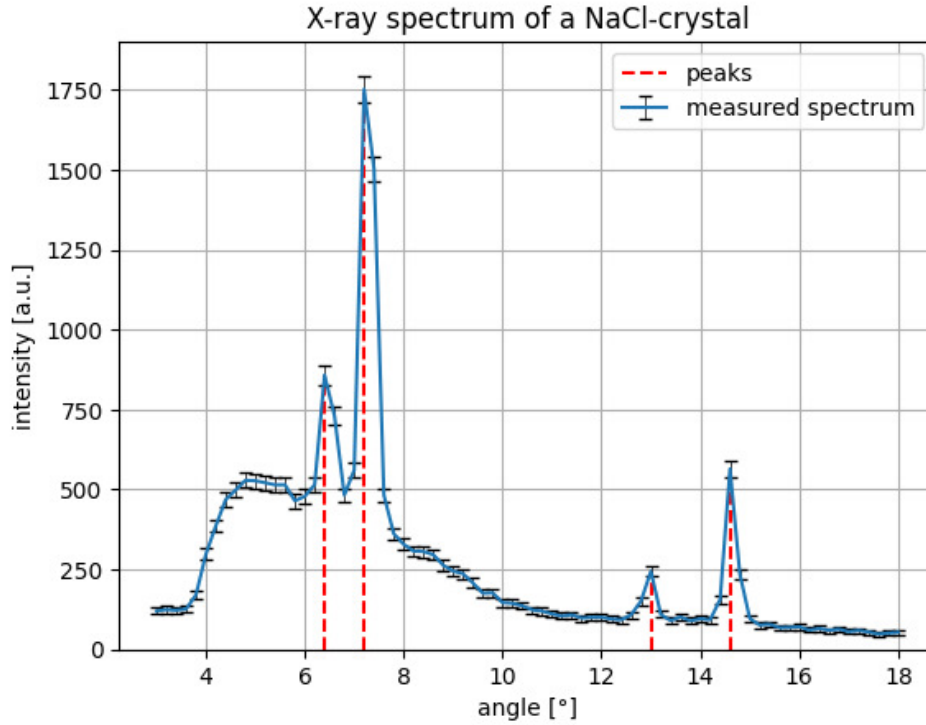


Figure 3.4: X-ray spectrum of Mo with LiF-crystal

We use `scipy_signal` to find the peaks and give them as starter μ -values to `curve_fit` and fit gaussians again. These angles $\vartheta = (6.5(3), 7.3(4), 12.92(15), 14.6(3))^\circ$ can be combined with their corresponding wavelengths $K_{\alpha,\beta}^{1,2} = (71.0(17), 71.1(10), 63.00(17), 63.2(10))$ pm from Abschnitt 3.2. ($K_{\alpha,\beta}$ lines with LiF) to calculate the lattice constant $a = 2d$ with NaCl via:

$$a = \frac{nK_{\alpha,\beta}}{\sin(\vartheta)} \quad \Delta a = a \sqrt{\frac{\Delta_K^2}{K^2} + \frac{\Delta_\theta^2}{\tan^2(\theta)}} \quad \boxed{a = (562 \pm 25) \text{ pm}} \quad (3.11)$$

The singular values can be seen in fig. 3.2, the last step was to calculate the mean, its error arises of the averaged singular errors Δa_i quadratically added with the standard deviation of all a_i .

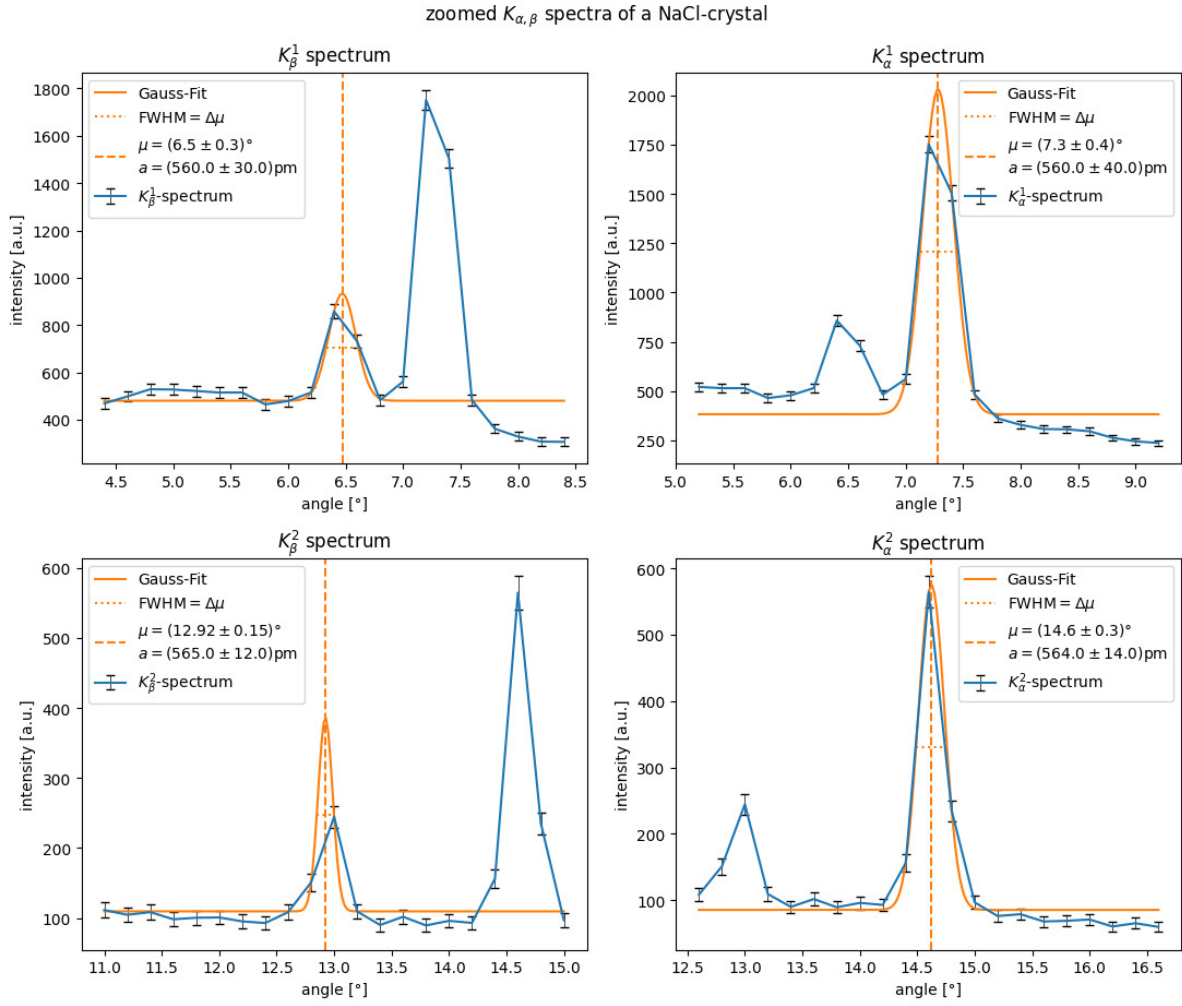


Figure 3.5: $K_{\alpha,\beta}$ Gaussian fits for NaCl

Avogadro constant: With given¹ values of $M_{\text{mol}} = 58.44 \text{ g mol}^{-1}$ and $\rho = 2.164 \text{ g cm}^{-3}$, as well as the previously determined a , eq. (1.6) can be used to calculate N_A :

$$N_A = \frac{4M_{\text{mol}}}{\rho a^3} \quad \Delta N_A = 12 \frac{M \Delta a}{a^4 \rho} \quad \boxed{N_A = (6.1 \pm 0.9) \cdot 10^{23} \text{ mol}^{-1}} \quad (3.12)$$

4. Discussion

4.1. spectrum

As one can see in fig. 3.1, the superposition of the continuous spectrum and characteristic lines (as well as their multiplicity - α and β peaks) can be observed *crystal* clear.

4.2. critical wavelength and critical angle

$\vartheta_{cr} = (9.7 \pm 2.3)^\circ$, that's where the 2nd order spectrum should start, but $\vartheta(K_\alpha^1) = (10.16 \pm 0.25)^\circ$. This can't be right, so I guess the error lies in the linear fit of fig. 3.1b.

4.3. Significance of h results

We are left with two different results for h according to the extrapolation of $I(\vartheta)$ or $I(U)$. Those methods are equivalent in the end¹. Both values show good non-significant deviation. Surprisingly though, contrary to the description in the script, the U measurement shows a more inaccurate result. This can be traced back most likely to an inaccurate linear fit in fig. 3.4. A steeper fit would have increased U_a and thus increased h .

Table 4.1: comparison of $h_{\vartheta,U}$ and h_{lit} ²

method	$h_{exp}[10^{-34}\text{J s}]$	$h_{lit}[10^{-34}\text{J s}]$	abs.Abw. $[10^{-34}\text{J s}]$	proz.Abw. $[\%]$	$S[\sigma]$
ϑ	6.5 ± 1.5	6.62607015	0.1 ± 1.5	2 ± 24	0.09
U	6.3 ± 0.6	6.62607015	0.3 ± 0.6	5 ± 10	0.6

4.4. $K_{\alpha,\beta}$ lines calculation with LiF

The script provides values for $K_{\alpha,\beta}$ of molybdenum, which can be used to check our experiment results, apparently they are perfect. But we need to talk about the gaussian fits: They rely on 4 parameters, but are only based on ~ 8 measured points. For a future experiment, one should decrease the goniometer's angle-step for this measurement and use a wider interval around the peak, so the gaussian base/underground can be fitted better.

Table 4.2: comparison of $K^{exp}(\text{Mo})$ and $K^{Lit}(\text{Mo})$ ¹

method	$K_\alpha^{exp}(\text{Mo})[\text{pm}]$	$K_\alpha^{lit}(\text{Mo})[\text{pm}]$	abs.Abw. $[\text{pm}]$	proz.Abw. $[\%]$	$S[\sigma]$
K_α	71.1 ± 1.4	71.1	0	0	0
K_β	63.1 ± 1.4	63.1	0	0	0

I fused the results for $n = 1, 2$ to one mean value by using their respective errors and standard deviation

$$\bar{K} = \text{mean}(K^1, K^2) \quad \Delta\bar{K} = \sqrt{\text{mean}(\Delta K^1, \Delta K^2)^2 + \text{std}(K_1, K_2)^2} \quad (4.1)$$

FHWM and ΔK : Of course one can argue, that ΔK is too high, because instead of variance σ^2 or standard deviation σ , we used the FWHM in fig. 3.2. But in a previous experiment, the script¹ told us to use FWHM as the error of peaks. Nonetheless though, the values for $K_{\alpha,\beta}$ are consistent through first and second order.

4.5. starting voltage

I'm confused, why the graphic approach yields different results for ΔU_a compared to using:

$$U_a = \frac{-c}{m} \quad \Delta U_a = U_a \sqrt{\frac{\Delta_c^2}{c^2} + \frac{\Delta_m^2}{m^2}} \quad \boxed{\Delta U_a = (22.3 \pm 1.3) \text{ kV}} \quad (4.2)$$

Please let me now, which method we should use for a future analysis.

4.6. NaCl crystal examination

When calculating $a_i = \frac{nK_{\alpha,\beta}}{\sin \vartheta}$, one could have used the mean values of the previously calculated $K_{\alpha,\beta}^n$ (e.g. average the two values for $K_{\alpha}^{1,2}$). Instead, I paired each ϑ with its corresponding K -line.

One can compare the result with its literature value³, the deviation is minimal.

Table 4.3: comparison of a_{exp} and a_{lit} ³

$a_{\text{exp}}[\text{pm}]$	$a_{\text{lit}}[\text{pm}]$	abs.Abw.[pm]	proz.Abw.[%]	$S[\sigma]$
562 ± 25	564.02	2 ± 25	0 ± 5	0.09

But, as one can see in fig. 3.5, the values for a differ dependent on n : $a_{n=2} > a_{n=1}$. This can't originate in the small angle approximation, because I only used it for the first task, probably it originates in a different refracting effect that changes with a higher inclination/angle of the crystal plate, but that's only a guess.

gaussian fits: Once again, but this time more extreme: The fits require 4 parameters, but the peaks itself consist of only 3! measured points, only the wide underground would superficially improve χ^2 . Clearly, we need a better resolution (at least angle step = 0.1° as in task 2).

4.7. Avogadro constant

This deviation is also negligible.

Table 4.4: comparison of N_A^{exp} and N_A^{lit} ⁴

$N_A^{\text{exp}}[\text{mol}^{-1}]$	$N_A^{\text{lit}}[\text{mol}^{-1}]$	abs.Abw. $[\text{mol}^{-1}]$	proz.Abw. $[\%]$	$S[\sigma]$
6.1 ± 0.9	6.02214076	0.1 ± 0.9	1 ± 15	0.09

I used wikipedia as a source for h and N_A because the values given in the script are not exact (the script itself references a source originating in 1999, which is outdated compared to the SI-system's reform in 2019).

4.8. conclusion

I'm a bit confused which method is best to determine the errors of those intersection points in the linear fits. It was quite interesting to fit those gaussians even though it wasn't necessarily required, but I discovered some nice python things by doing it. The experiment itself was satisfying because most values were automatically determined and saved. Finally, it's pretty nice actually using crystals and bragg's law to our advantage.

Literature references

- ¹Dr. J. Wagner, *Physikalisches praktikum 2.2 für studierende der physik b.sc.* [Online; Stand 04/2026] (Universität Heidelberg, 04/2026), https://git.physi.uni-heidelberg.de/schwierz/Versuchsanleitungen/src/branch/main/PAP2_2.pdf (visited on 04/14/2026) (cit. on pp. 3–7, 12, 14, 19–21).
- ²Wikipedia, *Planck constant*, [Online; Stand 07/2026], https://en.wikipedia.org/wiki/Planck_constant (cit. on p. 20).
- ³Wikipedia, *Sodium chloride (lattice constant)*, [Online; Stand 07/2026], https://en.wikipedia.org/wiki/Sodium_chloride (cit. on p. 21).
- ⁴Wikipedia, *Avogadro constant*, [Online; Stand 07/2026], https://en.wikipedia.org/wiki/Avogadro_constant (cit. on p. 22).

Figure references

- ⁵City council of Pripyat, *Pripyat evacuation announcement*, [Online; Stand 05/2026], <https://www.youtube.com/watch?v=P12i0G29-PI> (cit. on p. 1).
- ⁶Wikipedia, *Iso 7010*, [Online; Stand 05/2026], https://de.wikipedia.org/wiki/ISO_7010 (cit. on p. 1).

A. Python-code

July 14, 2026

```
[339]: # Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

from decimal import Decimal, ROUND_CEILING, ROUND_HALF_UP, getcontext #L
↳ Besonders für sig. Runden
import math

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
import scipy.constants as cc
from scipy.stats import norm
from scipy.optimize import curve_fit
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.integrate import quad
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelemin, argrelemax
from scipy.special import factorial
import scipy.integrate as integrate
from scipy.special import gamma

%matplotlib widget
import matplotlib.pyplot as plt
from matplotlib import colormaps
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path
from pathlib import Path
```



```

import pandas as pd # Auch wichtig für den Latex Export
from pytexit import py2tex
import csv
import re
from typing import List

# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars

# Display und Output
from IPython.display import display, Math, Latex, HTML

import textwrap

# Grafiken in EU-Größen ausgeben
def figsize_cm(width_cm, height_cm):
    return (2*width_cm / 2.54, 2*height_cm / 2.54)
def comma_to_float(valstr):
    return float(valstr.replace(',', '.'))

g=9.80984
Delta_g=0.00002

```

```

[340]: currentversuch = "255"
base = Path.cwd()
Messwerte_path = Path(base.parent.parent.parent/"Messwerte"/currentversuch)
↳ # initializing paths
Ausgabe_path = Path(base/"Diagramme")
print(Messwerte_path)
print(Ausgabe_path)

```

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Messwerte\255

c:\Users\samue\OneDrive\Dokumente\PAP\PAP2\Auswertungen2.2\Code\255\Diagramme

Delete UTF8 False encoding

```

[341]: # def deletevocals(text):
#     text=textwrap.dedent(text)
#     patterna = r'[][a] '
#     text=re.sub(patterna, "ä", text)
#     patternu=r'[][u] '
#     text=re.sub(patternu, "ü", text)
#     patterno=r'[][o] '
#     text=re.sub(patterno, "ö", text)
#     return text

```

```

# Regexpwissen: [0-9], [A-Z] findet egal welchen Zahl/Buchstaben; [^0-9] findet
↳ erstes Zeichen, das keine Zahl ist;

def deletevowels(text):
    # Einrückungen entfernen (gemeinsame Tabs des gesamten Textes)
    text = textwrap.dedent(text)
    # entfernt Bindestriche: - findet Bindestrich, \n Zeilenumbruch direkt
↳ dahinter, \s Whitespaces *beliebig viele
    text = re.sub(r'-\n\s*', "", text)
    # entfernt Zeilenumbrüche
    text = text.replace("\n", " ")

    # Eine Funktion, die vom re.sub aufgerufen wird, wenn ein Treffer erzielt
↳ wird
    def convert(match):
        # match.group(1) ist der Buchstabe nach dem Pünktchen (z.B. 'a' oder
↳ 'A')
        buchstabe = match.group(1)
        # Dictionary für die echten Umlaute
        umlaute = {'a': 'ä', 'u': 'ü', 'o': 'ö', 'A': 'Ä', 'U': 'Ü', 'O': 'Ö'}
        # Gib den Umlaut zurück. Falls das Zeichen nicht im Dict ist, lass es
↳ wie es ist
        return umlaute.get(buchstabe, buchstabe)

    # Das Pattern sucht nach " gefolgt von einem beliebigen Buchstaben aus der
↳ Auswahl, [X] findet genau ein Zeichen aus Menge X
    return re.sub(r'"([auoAUO])'", convert, text)

```

```

[342]: print(deletevowels("""
"""))

```

Runden signifikanter Stellen

```

[343]: def round_to_sigs(val, errVal=None, einheiten=None):
    """
    Funktion zur Rundung eines Fehlers und die Anpassung des Messwertes daran..
↳ Diese Funktion wurde etwas umständlicher
    geschrieben, da python mit Floats und Runden schnell in Probleme rennt..
↳ Daher musste hier mit dezimal gearbeitet werden.
    Zudem sollten besonders kleine und große Messwerte in der
↳ Dezimalschreibweise geschrieben werden, damit diese auch
    für Protokolle geeignet sind.

    Parameter

```

```

-----
**val** : float
    Messwert

**errVal** : float
    Ungenauigkeit des Messwertes

Return
-----

**value_rounded** : str
    Gerundeter Messwert

**error_rounded** : str
    Gerundete Ungenauigkeit des Messwertes

**res** : str
    "Messwert \pm Fehler"
"""
einheiten_str = einheiten if einheiten is not None else ""
if errVal is not None:
    # Daten zu Dezimal wechseln, da Floats probleme machen
    val = Decimal(str(val))
    errVal = Decimal(str(errVal))

    exp = int(math.floor(math.log10(float(errVal)))) # Die erste
    ↪signifikante Stellenposition wird anhand des errVal bestimmt

    if round(float(errVal / (Decimal(10) ** exp))) < 3: # wenn
    ↪1,2,3 erste signifikante Stelle, dann kommt eine zweite
    ↪Nachkommastellenposition hinzu
        exp -= 1

    scale = Decimal(10) ** exp

    val_round = (val / scale).quantize(Decimal('1'),
    ↪rounding=ROUND_HALF_UP) * scale # mit scale wird das Komma so
    ↪verschoben, dass alle signifikanten Stellen vor dem Komma stehen, und dann
    ↪alle Nachkommastellen weggerundet werden
    err_round = (errVal / scale).quantize(Decimal('1'),
    ↪rounding=ROUND_CEILING) * scale

    num = f"\num{{{val_round}}({err_round})}"
    qty = f"\qty{{{val_round}}({err_round)}}{{{einheiten_str}}}" if
    ↪einheiten else num

    return float(val_round), float(err_round), num, qty
else:

```

```

    val = Decimal(str(val))
    exp = int(math.floor(math.log10(float(val))))
    if round(float(val / (Decimal(10) ** exp))) < 3:           # wenn 1,2,3
↳ erste signifikante Stelle, dann kommt eine zweite Nachkommastellenposition
↳ hinzu
        exp -= 1
    scale = Decimal(10) ** exp
    val_round = (val / scale).quantize(Decimal('1'),
↳ rounding=ROUND_HALF_UP) * scale
    num = f"\\num{{{val_round}}}"
    qty = f"\\qty{{{val_round}}}{einheiten_str}" if einheiten else num
    return float(val_round), None, num, qty

v_round = np.vectorize(round_to_sigs, otypes=[float, float, object, object],
↳ excluded=['einheiten'])

# wissenschaftliche Notation erstmal vernachlässigen (da häufig Einheiten
↳ gewollt sind, die 10~/e Präfixe beinhalten)

```

Gauss'sche Fehlerfortpflanzung

```

[344]: def gff(function, errPronePar, latex=True):
    """
    Kann die Fehlerformel einer gegebenen Gleichung bestimmen.

    Parameters
    -----
    **func** : sympy function
        Funktion dessen Fehler bestimmt werden soll.

    **errPronePar** : Array
        Liste (Array) aller fehlerbehafteten Größen der Gleichung con sp.Symbols
        Diese Werte werden als x_sym, y_sym, z_sym etc. bezeichnet und sind
↳ ungleich den Werten für x, y, z.
        Für die Werte wird daher die Bezeichnung x_val, y_val, z_val etc.
↳ genutzt und für deren Fehler err_x, err_y, err_z etc.

    Return
    -----
    **absolut_err** : sympy function
        Gibt die Fehlergleichung des absoluten Fehlers wieder.

    **relativ_err** : sympy function
        Gibt die Fehlergleichung des relativen Fehlers wieder.

    **errProneParamters** : array
        Liste aller Fehlerbehafteten Größen
    """

```

```

"""

error = 0
errProneParamaters = []

function = sp.simplify(function)
variables = errPronePar.split(",")
variables = sp.symbols(variables)

for variable in variables:
    Delta = sp.Symbol(f"Delta_{variable}")
    partial = sp.diff(function, variable)           # Die Funktion
     $\rightarrow$  wird nach der fehlerbehafteten Variable abgeleitet
    term=sp.UnevaluatedExpr(partial*Delta)**2
    error = error + ((term))                       # Fehler werden
     $\rightarrow$  quadratisch aufsummiert
    errProneParamaters.append((variable,Delta))

absolute_err=sp.simplify(sp.sqrt(error),rational = True)
relative_err=sp.simplify(sp.sqrt(error/function**2),rational = True)

if latex:
    #Prints out the Latex Code for the Functions
    print("Funktion:")
    display(Math(sp.latex(function,long_frac_ratio=2)))
    print(sp.latex(function))
    print("Absoluter Fehler:")
    display(Math(sp.latex(absolute_err,long_frac_ratio=2)))
    print(sp.latex(absolute_err))
    print("Relativer Fehler:")
    display(Math(sp.latex(relative_err,long_frac_ratio=2)))
    print(sp.latex(relative_err))
    return function,absolute_err, relative_err, errProneParamaters

```

Sigma-Abweichungen

```

[345]: def sigma_abweichung(p1, err_p1,p2,err_p2):
        """
        Funktion zum berechnen der Sigma-Abweichugn von zwei Messwerten, oder einem
         $\rightarrow$  Messwert und einem Literaturwert.
        """
        if err_p1 == 0 and err_p2 == 0:
            raise ValueError("Für die Sigma-Abweichung muss mindestens ein Wert
             $\rightarrow$  fehlerbehaftet sein!")
        else:
            abweichung=Decimal(str(abs(p1 - p2)/(np.sqrt(err_p1 ** 2+err_p2 ** 2))))

```

```

if abweichung == 0:
    return 0.0, "\\num{0}"

exp = int(math.floor(math.log10(float(abweichung))))
if round(float(abweichung / (Decimal(10) ** exp))) < 3:           # wenn
↪ 1,2,3 erste signifikante Stelle, dann kommt eine zweite
↪ Nachkommastellenposition hinzu
    exp -= 1
    scale = Decimal(10) ** exp
    abweichung_round = (abweichung / scale).quantize(Decimal('1'),
↪ rounding=ROUND_CEILING) * scale
    res = f"\\num{{{abweichung_round}}}"

return float(abweichung_round), res

```

```

[346]: # #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2
#     elif val1>val2:
#         return name1+">"+name2
#     else:
#         return name1+"="+name2

#Erstellung von Vergleichstabellen in Latex
def compare_table_array(nam1, nam2, einheiten, val1_array, err1_array,
↪ val2_array, err2_array):
    # 1. Tabellenkopf (Hier werden die Strings einmalig eingesetzt)
    output_header = textwrap.dedent(f"""
        \\begin{{table}}[htbp]
        \\centering
        \\caption{{Vergleich von ${nam1}$ und ${nam2}$}}
        \\begin{{tabularx}}{{0.8\\textwidth}}{{ZZZZZ}}
        \\toprule
        Messreihe & {nam1}[\\unit{{{einheiten}}}] &
↪ {nam2}[\\unit{{{einheiten}}}] & \\text{{abs.Abw.}}[\\unit{{{einheiten}}}] &
↪ \\text{{proz.Abw.}}[\\unit{{\\percent}}] & S[\\sigma] \\midrule
        """).strip() # strip macht whitespaces am anfang und ende des strings weg

    table_rows = []

    # 2. Der Body (Schleife läuft über die Werte-Arrays)
    for val1, err1, val2, err2 in zip(val1_array, err1_array, val2_array,
↪ err2_array):

        VAL1=round_to_sigs(val1,err1,r')[2]
        if err2!=0:

```

```

        VAL2=round_to_sigs(val2,err2,r'[2]
else:
    VAL2=val2

abs=np.abs(val1-val2)
abs_delta=np.sqrt(err1**2+err2**2)
if abs_delta!=0:
    SIGMA=sigma_abweichung(val1,err1,val2,err2)[1]
else:
    SIGMA=0.0

rel=abs/np.abs(val1)*100
if abs != 0:
    rel_delta = rel * np.sqrt((abs_delta / abs)**2 + (err1 / val1)**2)
    ABS = round_to_sigs(abs, abs_delta, r'[2]
    REL = round_to_sigs(rel, rel_delta, r'[2]
else:
    rel_delta = 0
    ABS = "0"
    REL = "0"

# Eine saubere Zeile nur mit den Werten für LaTeX
row = f"          & {VAL1} & {VAL2} & {ABS} & {REL} & {SIGMA} \\\\"
table_rows.append(row)

body_str = "\\n".join(table_rows)

# 3. Tabellenfuß
output_footer = textwrap.dedent(f"""
    \\bottomrule
    \\end{{tabularx}}
    \\label{{table:Vergleich_{{nam1}}}}
    \\end{{table}}
""").strip()

# Alles zusammensetzen
final_output = f"{output_header}\\n{body_str}\\n{output_footer}"

print(final_output)

```

```

[347]: # #Erstellung von Vergleichstabellen in Latex
# def compare_table_withliterature(nam1,nam2,einheiten,val1,err1,val2):
#     VAL1=round_to_sigs(val1,err1,r'[2]
#     abs=np.abs(val1-val2)
#     abs_delta=np.sqrt(err1**2)
#     if abs_delta!=0:
#         SIGMA=sigma_abweichung(val1,err1,val2,0)[1]

```

```

#     else:
#         SIGMA=0.0
#         ABS=round_to_sigs(abs,abs_delta,r')[2]
#         rel=abs/np.abs(val2)*100
#         rel_delta=rel*np.sqrt((abs_delta/abs)**2+(err1/val1)**2) if abs != 0 else 0
#     REL=round_to_sigs(rel,rel_delta,r')[2]
#     output = textwrap.dedent(f"""
#         \\begin{{table}}[htb]
#         \\centering
#         \\caption{{}}
#         \\begin{{tabularx}}{{0.8\\textwidth}}{{ZZZZZ}}
#         \\toprule
#         \\
#         {\\nam1}[\\unit{{{{einheiten}}}}]&{\\nam2}[\\unit{{{{einheiten}}}}]&\\text{{abs.Abw.
#         }}[\\unit{{{{einheiten}}}}]&\\text{{proz.Abw.
#         }}[\\unit{{\\percent}}]&S[\\sigma]\\\\\\\\\\midrule
#         {VAL1}&\\num{{{{val2}}}}&{ABS}&{REL}&{SIGMA}\\\\\\\\
#         \\bottomrule
#         \\end{{tabularx}}
#         \\label{{table:Vergleich_{{nam1}}}}
#         \\end{{table}}
#     """)
#     print(output)

# #Größenvergleich in latex
# def size_comp_str(name1,name2,val1,val2):
#     if val1<val2:
#         return name1+"<"+name2
#     elif val1>val2:
#         return name1+">"+name2
#     else:
#         return name1+"="+name2

#Erstellung von Vergleichstabellen in Latex
import textwrap
def compare_table(nam1,nam2,einheiten,val1,err1,val2,err2):
    VAL1=round_to_sigs(val1,err1,r')[2]
    if err2!=0:
        VAL2=round_to_sigs(val2,err2,r')[2]
    else:
        VAL2=val2
    abs=np.abs(val1-val2)
    abs_delta=np.sqrt(err1**2+err2**2)
    if abs_delta!=0:

```



```

        SIGMA=sigma_abweichung(val1,err1,val2,err2)[1]
    else:
        SIGMA=0.0

    rel=abs/np.abs(val1)*100
    if abs != 0:
        rel_delta = rel * np.sqrt((abs_delta / abs)**2 + (err1 / val1)**2)
        ABS = round_to_sigs(abs, abs_delta, r')[2]
        REL = round_to_sigs(rel, rel_delta, r')[2]
    else:
        rel_delta = 0
        ABS = "0"
        REL = "0"

    output = textwrap.dedent(f"""
        \\begin{{table}}[htbp]
        \\centering
        \\caption{{Vergleich von ${nam1}$ und ${nam2}$}}
        \\begin{{tabularx}}{{\\textwidth}}{{ZZZZx}}
        \\toprule
        \\
        {nam1}[\\unit{{einheiten}}]&{nam2}[\\unit{{einheiten}}]&\\text{{abs.Abw.
        }}[\\unit{{einheiten}}]&\\text{{proz.Abw.
        }}[\\unit{{percent}}]&S[\\sigma]\\\\\\midrule
        {VAL1}&{VAL2}&{ABS}&{REL}&{SIGMA}\\\\
        \\bottomrule
        \\end{{tabularx}}
        \\label{{table:Vergleich_{nam1}}}
        \\end{{table}}
    """)
    print(output)

```

```

[364]: angle,intensity = np.loadtxt(Messwerte_path/"A1_30_06_2026_A1.txt", skiprows=1,
    converters={0:comma_to_float, 1:comma_to_float},unpack=True) # Intensität
    pro Winkel
    Delta_intensity=np.sqrt(intensity)
    def linear(x,m,a):
        return m*x+a
    popt, pcov=curve_fit(linear, angle[10:17],intensity[10:17])
    plt.close('all')

    plt.
    errorbar(angle,intensity,yerr=Delta_intensity,fmt='-',linewidth=3,capsize=3,elinewidth=0.
    5,ecolor='black',label='measured intensity values')
    plt.errorbar(angle[10:17],intensity[10:17],yerr=Delta_intensity[10:
    17],linestyle='dashed',color='lightcyan',ecolor='black',capsize=3,elinewidth=0.
    5,label='fit points')

```

```

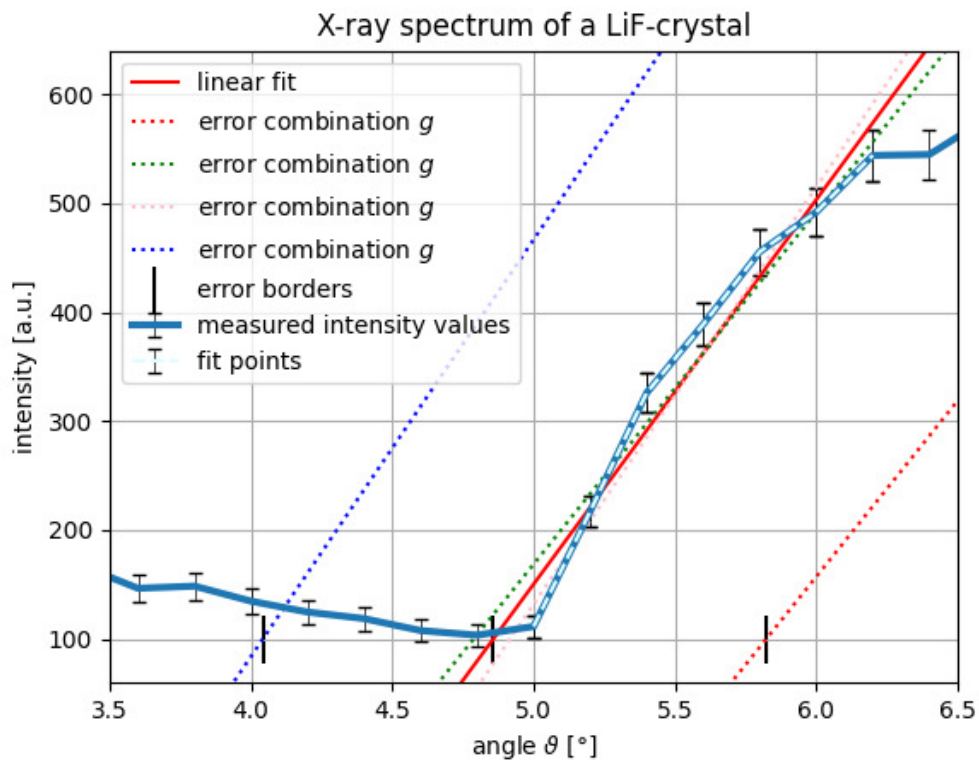
plt.plot(x, linear(x,*popt),color='red',label='linear fit')

m_1=popt[0]-np.sqrt(pcov[0][0])
m_2=popt[0]+np.sqrt(pcov[0][0])
c_1=popt[1]-np.sqrt(pcov[1][1])
c_2=popt[1]+np.sqrt(pcov[1][1])
x=np.linspace(3.5,6.5,10)
plt.plot(x, linear(x,m_1,c_1),linestyle='dotted',color='red',label=r'error_␣
↪combination $g$')
plt.plot(x, linear(x,m_1,c_2),linestyle='dotted',color='green',label=r'error_␣
↪combination $g$')
plt.plot(x, linear(x,m_2,c_1),linestyle='dotted',color='pink',label=r'error_␣
↪combination $g$')
plt.plot(x, linear(x,m_2,c_2),linestyle='dotted',color='blue',label=r'error_␣
↪combination $g$')
points=[(100-c_1)/m_1,(100-c_2)/m_2,(100-popt[1])/popt[0]]
plt.scatter(points,[100,100,100],marker='|',s=400,color='black',label='error_␣
↪borders')
theta=points[2]
Delta_theta=np.max([np.abs(points[0]-points[2]),np.abs(points[2]-points[1])])
theta,Delta_theta,_,latexUa=round_to_sigs(theta,Delta_theta,r'\degree')
print(latexUa)

plt.xlabel(r'angle $\vartheta$ [°]')
plt.ylabel(r'intensity [a.u.]')
plt.title(r'X-ray spectrum of a LiF-crystal')
plt.xlim(3.5,6.5)
plt.ylim(60,640)
# plt.text(0.8, 0.2,rf'$E_R=\{E_R\}(\{Delta_E_R\})$keV',␣
↪horizontalalignment='center',verticalalignment='center', transform=plt.gca().
↪transAxes)
# plt.text(0.8, 0.1,rf'$\sigma_{\{12\}}=\{\sigma\}(\{Delta_\sigma\})$',␣
↪horizontalalignment='center',verticalalignment='center', transform=plt.gca().
↪transAxes)
plt.legend()
plt.grid()
plt.show()
plt.savefig(Ausgabe_path/"task1_LiFspectrumzoomed.png",format="png")

```

\qty{4.9(1.0)}{\degree}



```
[349]: critang,Delta_critang,_,latexcrit=round_to_sigs(np.radians(theta),np.
        ↪radians(Delta_theta))
print(critang,Delta_critang)
n=1
d = 201.4*1e-12
critlambda = 2*d*critang / n
Delta_critlambda = 2*d*Delta_critang / n
critlambda,Delta_critlambda,_,latexLcr=round_to_sigs(critlambda,Delta_critlambda,r'\nano\m')
print(latexLcr)

U_a=35*1e3
Delta_U_a=0.05*1e3
h=U_a*cc.e*critlambda/cc.c
Delta_h=h*np.sqrt(Delta_U_a**2/U_a**2 + Delta_critlambda**2/critlambda**2)
h,Delta_h,_,latexh=round_to_sigs(h,Delta_h,r'\joule\s')
print(latexh)

n=2
theta_2=n*critlambda/(2*d)
Delta_theta_2=np.sqrt(Delta_critlambda**2*n**2/d**2)/2
```

```

theta_2,Delta_theta_2,_,latextheta_2=round_to_sigs(theta_2,Delta_theta_2,r'\radian')
_,_,_,latexTHETA_2=round_to_sigs(np.degrees(theta_2),np.
    ↳degrees(Delta_theta_2),r'')
print(latextheta_2)
print(latexTHETA_2)

```

```

0.086 0.018
\qty{3.5E-11(8E-12)}{\nano\m}
\qty{6.5E-34(1.5E-34)}{\joule\s}
\qty{0.17(0.04)}{\radian}
\num{9.7(2.3)}

```

```

[362]: peaks={"A2_Ka1_30_06_2026_14_52_19.txt":(r"$K^1_\alpha$",10.
    ↳15),"A2_Kb1_30_06_2026_14_52_19.txt":(r"$K^1_\beta$",9),"A2_Ka2_30_06_2026_
    ↳14_52_19.txt":(r"$K^2_\alpha$",20.65),"A2_Kb2_30_06_2026_14_52_19.txt":
    ↳(r"$K^2_\beta$",18.3)}
K=[]
Delta_K=[]
plt.close('all')
fig, axes = plt.subplots(2, 2, figsize=(figsize_cm(14,12)))
fig.suptitle(r'zoomed $K_{\alpha,\beta}$ spectra of a LiF-crystal')

def gaussian(x, A,mu, sig,c): #A: Flaeche der Gaussfunktion
    return A/(sqrt(2*pi)*sig)*exp(-(x-mu)**2/2/sig**2)+c

for ax,(pfad,info) in zip(axes.flatten(),peaks.items()):
    name = info[0]
    mu_start = info[1]

    angle,intensity= np.loadtxt(Messwerte_path/pfad, converters={0:
    ↳comma_to_float, 1:comma_to_float},unpack=True)
    Delta_intensity=np.sqrt(intensity)

    ax.
    ↳errorbar(angle,intensity,yerr=Delta_intensity,fmt='-',capsize=3,elinewidth=0.
    ↳5,ecolor='black',label=f"{name}-spectrum")

    c_start = np.min(intensity) # Untergrundrauschen
    A_start =(np.max(intensity) - c_start) * (0.1 * np.sqrt(2 * np.pi)) #
    ↳Fläche rückwärts berechnet
    popt, pcov=curve_fit(gaussian,angle,intensity,p0=[A_start,mu_start,0.
    ↳1,c_start],sigma=Delta_intensity,absolute_sigma=True)
    x=np.linspace(angle[0],angle[-1],100)
    ax.plot(x, gaussian(x,*popt), label='Gauss-Fit')

    FWHM=2*popt[2]*np.sqrt(2*np.log(2))
    mu_sig,Delta_mu_sig,_,latexmusig=round_to_sigs(popt[1],FWHM,r'\degree')

```

```

    ax.hlines(1/2*(gaussian(popt[1],*popt)-popt[3])+popt[3],xmin=popt[1]-FWHM/
↪2,xmax=popt[1]+FWHM/2,linestyle='dotted',color='C1',label=r'FWHM$=\Delta_{\mu}$')

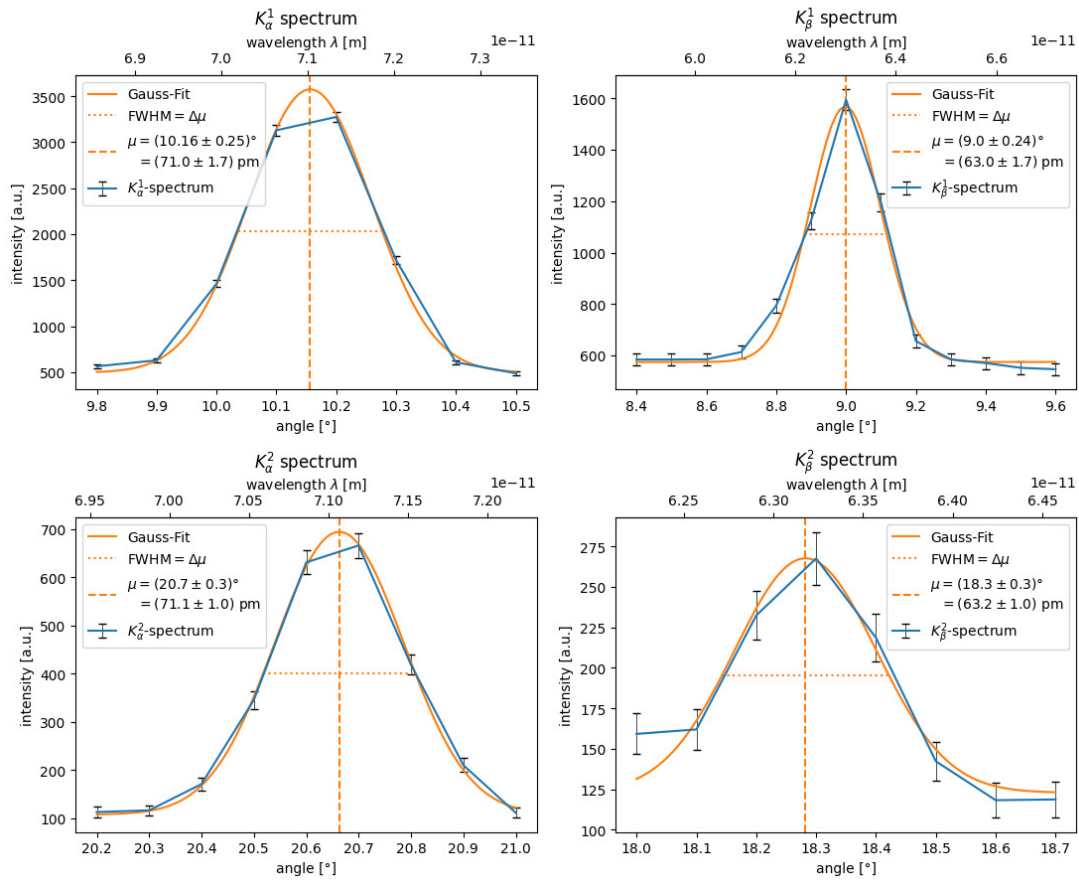
    ax.set_xlabel(r'angle [°]')
    ax.set_ylabel(r'intensity [a.u.]')

    def deg_to_wavelength(deg,n):
        theta_rad = np.radians(deg)
        return 2 * d * np.sin(theta_rad)/n
    def wavelength_to_deg(wl,n):
        # Arcussinus abfangen, falls Werte außerhalb des Definitionsbereichs liegen
        with np.errstate(invalid='ignore'):
            return np.degrees(np.arcsin(n*wl / (2 * d)))

    n_ord = 1 if popt[1] < 15 else 2
    ↪
    ↪Lambda,Delta_lambda,_,latexlambda=round_to_sigs(1e12*deg_to_wavelength(popt[1],n_ord),1e12*
↪2*FWHM,n_ord)-deg_to_wavelength(popt[1]-1/2*FWHM,n_ord)),r'\pico\m')
    K.append(Lambda)
    Delta_K.append(Delta_lambda)
    ax.
    ↪axvline(popt[1],linestyle='--',color='C1',label=rf'$\mu=({\mu\_sig}\pm{\Delta\_mu\_sig})^{\circ}$'+'\n'
↪\;\;,\;=({\Lambda}\pm{\Delta\_lambda})$ pm')
    fwd = lambda deg,n=n_ord: deg_to_wavelength(deg, n)
    inv = lambda wl,n=n_ord: wavelength_to_deg(wl, n)
    secax = ax.secondary_xaxis('top', functions=(fwd, inv))
    secax.set_xlabel(r'wavelength $\lambda$ [m]')
    ax.set_title(name+' spectrum')
    ax.legend()
plt.tight_layout()
plt.savefig(Ausgabe_path/'Klines.png',format="png",bbox_inches='tight')
plt.show()

```

zoomed $K_{\alpha,\beta}$ spectra of a LiF-crystal



```
[351]: print(sigma_abweichung(71,1.7,71.1,1.0))
print(sigma_abweichung(63,1.7,63.2,1.0))

mu_deg=10.16
sigma=0.1022
Delta_sigma=0.0016
FWHM_deg=2*np.sqrt(2*np.log(2))*sigma
Delta_FWHM_deg=2*np.sqrt(2*np.log(2))*Delta_sigma
fhw,Delta_fhw,_,Latexfhw=round_to_sigs((2 * d * np.cos(np.radians(mu_deg)) *
↳ np.radians(FWHM_deg)),(2 * d * np.cos(np.radians(mu_deg)) * np.
↳ radians(Delta_FWHM_deg)),r'\m')
print(Latexfhw)

(0.06, '\\num{0.06} ')
(0.11, '\\num{0.11} ')
\qty{1.67E-12(3E-14)}{\m}
```

[361]:

```
voltage=np.arange(20,36,1)
Delta_voltage=0.05
n=np.array([3.0,4.5,6.75,10.5,41.9,122.0,183.9,237.0,296.9,338.7,393.6,440.
↪5,483.4,521.4,573.1,609.1])
Delta_n=np.sqrt(n)
def linear(x,m,a):
    return m*x+a
popt, pcov=curve_fit(linear, voltage[5:],n[5:],sigma=Delta_n[5:
↪],absolute_sigma=True)
plt.close('all')

plt.
↪errorbar(voltage,n,xerr=Delta_voltage,yerr=Delta_n,fmt='-',linewidth=2,capsize=3,elinewidth
↪5,ecolor='black',label='measured rates')
plt.errorbar(voltage[5:],n[5:
↪],linestyle='dashed',color='lightcyan',ecolor='black',capsize=3,elinewidth=0.
↪5,label='fit points')
plt.plot(np.linspace(20,35,10), linear(np.
↪linspace(20,35,10),*popt),color='red',label=r'linear fit $f$')
m_1=popt[0]-np.sqrt(pcov[0][0])
m_2=popt[0]+np.sqrt(pcov[0][0])
c_1=popt[1]-np.sqrt(pcov[1][1])
c_2=popt[1]+np.sqrt(pcov[1][1])

plt.plot(np.linspace(20,35,10), linear(np.
↪linspace(20,35,10),m_1,c_1),linestyle='dotted',color='red',label=r'error_
↪combination $g$')
plt.plot(np.linspace(20,35,10), linear(np.
↪linspace(20,35,10),m_1,c_2),linestyle='dotted',color='green',label=r'error_
↪combination $g$')
plt.plot(np.linspace(20,35,10), linear(np.
↪linspace(20,35,10),m_2,c_1),linestyle='dotted',color='pink',label=r'error_
↪combination $g$')
plt.plot(np.linspace(20,35,10), linear(np.
↪linspace(20,35,10),m_2,c_2),linestyle='dotted',color='blue',label=r'error_
↪combination $g$')
[-c_1/m_1,-c_2/m_2,-popt[1]/popt[0]]
points=[-c_1/m_1,-c_2/m_2,-popt[1]/popt[0]]
plt.scatter(points,[0,0,0],marker='|',s=400,color='black',label='error borders')
U_a=points[2]
Delta_U_a=np.max([points[0]-points[2],points[2]-points[1]])
U_a,Delta_U_a,_,latexUa=round_to_sigs(U_a,Delta_U_a,r'kilo\volt')
print(latexUa)
m=popt[0]
c=popt[1]
Delta_m=np.sqrt(pcov[0][0])
```

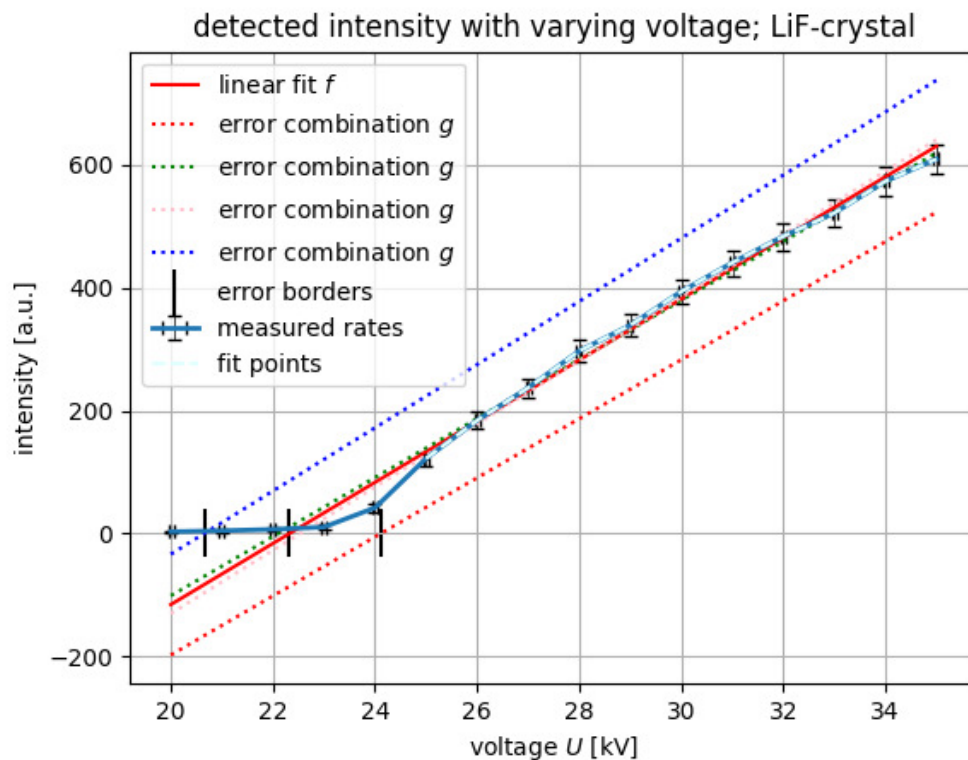
```

Delta_c=np.sqrt(pcov[1][1])
Delta_U_a= U_a*np.sqrt(Delta_c**2/c**2 + Delta_m**2/m**2)
U_a,Delta_U_a,,latexUa=round_to_sigs(U_a,Delta_U_a,r'kilo\volt')
print(latexUa)
plt.xlabel(r'voltage $U$ [kV]')
plt.ylabel(r'intensity [a.u.]')
plt.title(r'detected intensity with varying voltage; LiF-crystal')
# plt.text(0.8, 0.2,rf'$E_R=\{E_R\}(\{Delta_E_R\})$keV',␣
# ↪horizontalalignment='center',verticalalignment='center', transform=plt.gca().
# ↪transAxes)
# plt.text(0.8, 0.1,rf'$\sigma_{\{12\}}=\{\sigma\}(\{Delta_\sigma\})$',␣
# ↪horizontalalignment='center',verticalalignment='center', transform=plt.gca().
# ↪transAxes)
plt.legend()
plt.grid()
plt.show()
plt.savefig(Ausgabe_path/"task3.png",format="png")

```

\qty{22.3(1.8)}{kilo\volt}

\qty{22.3(1.3)}{kilo\volt}




```
[353]: critang,Delta_critang=np.radians(7.5),np.radians(0.1)
n=1
d = 201.4*1e-12
critlambda = 2*d*critang / n
Delta_critlambda = 2*d*Delta_critang / n
critlambda,Delta_critlambda,_,latexLcr=round_to_sigs(critlambda,Delta_critlambda,r'\nano\m')
print(latexLcr)

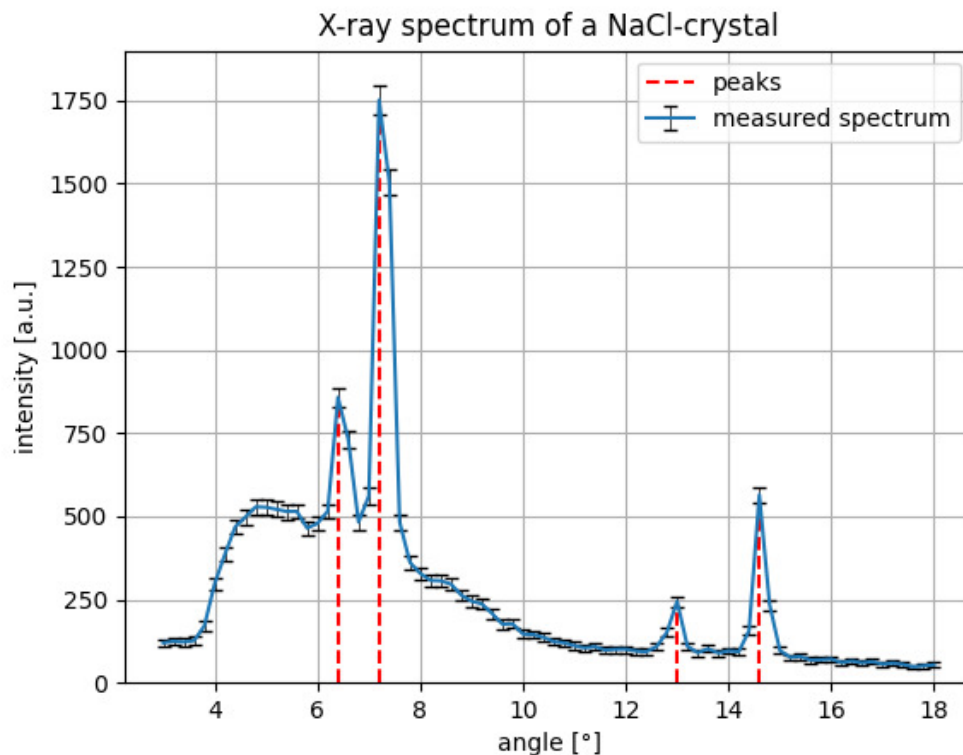
U_a=22.3*1e3
Delta_U_a=1.8*1e3
h=U_a*cc.e*critlambda/cc.c
Delta_h=h*np.sqrt(Delta_U_a**2/U_a**2 + Delta_critlambda**2/critlambda**2)
h,Delta_h,_,latexh=round_to_sigs(h,Delta_h,r'\joule\s')
print(latexh)
```

$\backslash\text{qty}\{5.27\text{E-}11(8\text{E-}13)\}\{\backslash\text{nano}\backslash\text{m}\}$

$\backslash\text{qty}\{6.3\text{E-}34(6\text{E-}35)\}\{\backslash\text{joule}\backslash\text{s}\}$

```
[360]: angle,intensity= np.loadtxt(Messwerte_path/"A4_30_06_2026 15_37_32.txt",
    ↪converters={0:comma_to_float, 1:comma_to_float},unpack=True)
Delta_intensity=np.sqrt(intensity)
peaks_indices,_=scipy.signal.find_peaks(intensity,prominence=150)
peak_angle=np.array(angle[peaks_indices])
# Im Plot markieren (z.B. mit grünen Dreiecken)

plt.close('all')
plt.title(r'X-ray spectrum of a NaCl-crystal')
plt.
    ↪vlines(peak_angle,ymin=0,ymax=intensity[peaks_indices],linestyle='dashed',color='red',label=
plt.
    ↪errorbar(angle,intensity,yerr=Delta_intensity,fmt='-',capsize=3,elinewidth=0.
    ↪5,ecolor='black',label=r'measured spectrum')
plt.xlabel(r'angle [°]')
plt.ylabel(r'intensity [a.u.]')
plt.ylim(0,1900)
plt.legend()
plt.grid()
plt.savefig(Ausgabe_path/'task4.png',format="png",bbox_inches='tight')
plt.show()
```



```
[355]: print(angle[peaks_indices[0]-2:peaks_indices[0]+3])
       print(K)
       print(Delta_K)
```

```
[6.  6.2 6.4 6.6 6.8]
[71.0, 63.0, 71.1, 63.2]
[1.7, 1.7, 1.0, 1.0]
```

```
[356]: angle,intensity= np.loadtxt(Messwerte_path/"A4_30_06_2026 15_37_32.txt",
    ↪converters={0:comma_to_float, 1:comma_to_float},unpack=True)
Delta_intensity=np.sqrt(intensity)
peaks_indices,_=scipy.signal.find_peaks(intensity,prominence=150)
peak_angle=np.array(angle[peaks_indices])

peaks=[r"$K^1\_beta$",r"$K^1\_alpha$",r"$K^2\_beta$",r"$K^2\_alpha$"]
K=np.array([63.0,71.0, 63.2, 71.1])
Delta_K=np.array([1.7, 1.7, 1.0, 1.0])

plt.close('all')
fig, axes = plt.subplots(2, 2, figsize=(figsize_cm(14,12)))
fig.suptitle(r'zoomed $K_{\alpha,\beta}$ spectra of a NaCl-crystal')
```

```

i=0
aarray=[]
Delta_aarray=[]
for ax,name in zip(axes.flatten(),peaks):
    indices=slice(peaks_indices[i]-10,peaks_indices[i]+11)
    anglearray=np.array(angle[indices])
    intensityarray=np.array(intensity[indices])
    Delta_intensityarray=Delta_intensity[indices]
    ax.
    errorbar(anglearray,intensityarray,yerr=Delta_intensityarray,fmt='-',capsize=3,elinewidth=0
    5,ecolor='black',label=f"{name}-spectrum")

    mu_start=angle[peaks_indices[i]]
    c_start = np.min(intensityarray) #
    Untergrundrauschen
    A_start =(np.max(intensityarray) - c_start) * (0.1 * np.sqrt(2 * np.pi))
    # Fläche rückwärts berechnet
    popt,
    pcov=curve_fit(gaussian,anglearray,intensityarray,p0=[A_start,mu_start,0.
    1,c_start],sigma=Delta_intensityarray,absolute_sigma=True)
    x=np.linspace(anglearray[0],anglearray[-1],200)
    ax.plot(x, gaussian(x,*popt), label='Gauss-Fit')

    FWHM=2*popt[2]*np.sqrt(2*np.log(2))
    mu_sig,Delta_mu_sig,_,latexmusig=round_to_sigs(popt[1],FWHM,r'\degree')
    print(latexmusig)
    ax.hlines(1/2*(gaussian(popt[1],*popt)-popt[3])+popt[3],xmin=popt[1]-FWHM/
    2,xmax=popt[1]+FWHM/2,linestyle='dotted',color='C1',label=r'FWHM$=\Delta$
    \mu$')

    ax.set_xlabel(r'angle [°]')
    ax.set_ylabel(r'intensity [a.u.]')

    # def deg_to_wavelength(deg,n):
    #     theta_rad = np.radians(deg)
    #     return 2 * d * np.sin(theta_rad)/n
    # def wavelength_to_deg(wl,n):
    #     # Arcussinus abfangen, falls Werte außerhalb des Definitionsbereichs
    #     # liegen
    #     with np.errstate(invalid='ignore'):
    #         return np.degrees(np.arcsin(n*wl / (2 * d)))

    n_ord = 1 if popt[1] < 10 else 2
    #
    Lambda,Delta_lambda,_,latexlambda=round_to_sigs(1e12*deg_to_wavelength(popt[1],n_ord),1e12*
    2*FWHM,n_ord)-deg_to_wavelength(popt[1]-1/2*FWHM,n_ord),r'\pico\m')

```

```

a=n_ord*K[i]/np.sin(np.radians(mu_sig))
Delta_a=a*np.sqrt(Delta_K[i]**2/K[i]**2 + np.radians(Delta_mu_sig)**2/np.
↪tan(np.radians(mu_sig))**2)
a,Delta_a,_,latexa=round_to_sigs(a,Delta_a,r'')
aarray.append(a)
Delta_aarray.append(Delta_a)
print(latexa)
ax.
↪axvline(popt[1],linestyle='--',color='C1',label=rf'$\mu=({mu\_sig})\pm{Delta\_mu\_sig})^\circ$'+'\n'

# fwd = lambda deg,n=n_ord: deg_to_wavelength(deg, n)
# inv = lambda wl,n=n_ord: wavelength_to_deg(wl, n)
# secax = ax.secondary_axis('top', functions=(fwd, inv))
# secax.set_xlabel(r'wavelength $\lambda$ [m]')
ax.set_title(name+' spectrum')
ax.legend()
i=i+1
a,Delta_a=np.mean(np.array(aarray)),np.sqrt(np.mean(np.
↪array(Delta_aarray))**2+np.std(np.array(aarray))**2)
a,Delta_a,_,latexa=round_to_sigs(a,Delta_a,r'\pico\m')
print(latexa)
plt.tight_layout()
plt.savefig(Ausgabe_path/'KlinesNaCl.png',format="png",bbox_inches='tight')
plt.show()

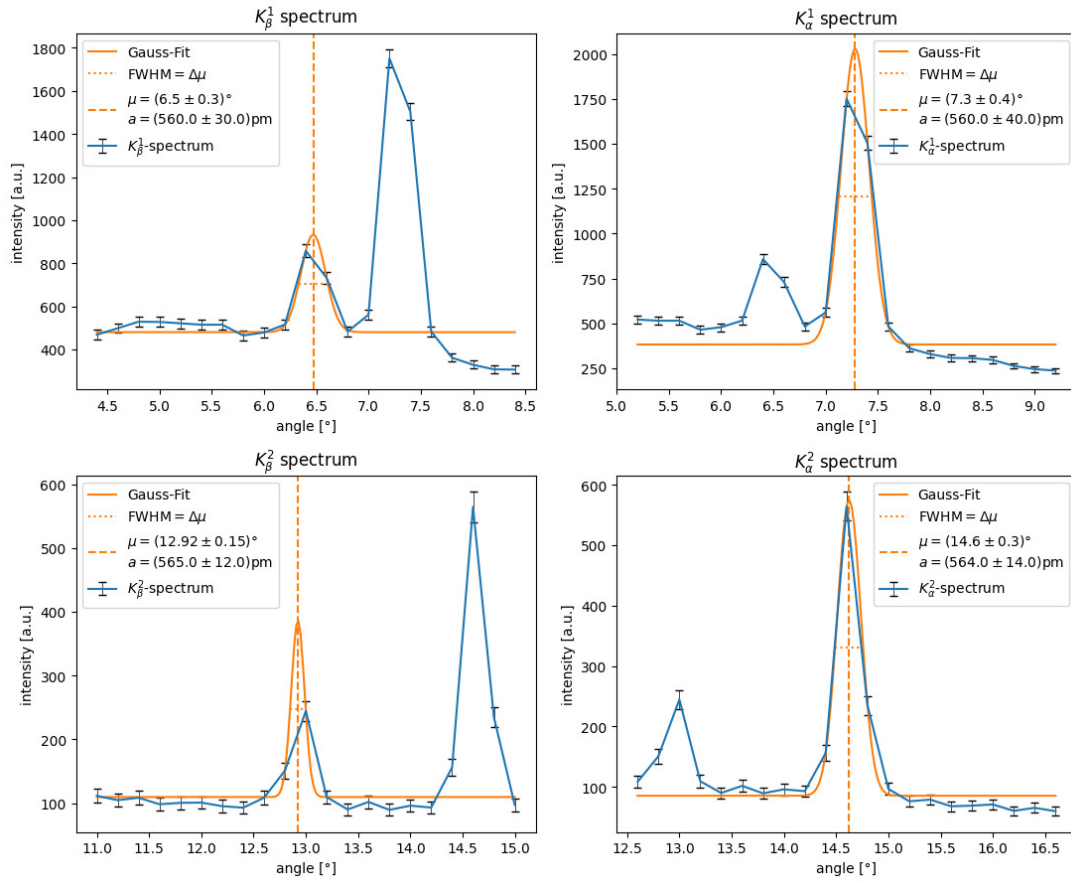
```

```

\qty{6.5(0.3)}{\degree}
\num{560(30)}
\qty{7.3(0.4)}{\degree}
\num{560(40)}
\qty{12.92(0.15)}{\degree}
\num{565(12)}
\qty{14.6(0.3)}{\degree}
\num{564(14)}
\qty{562(25)}{\pico\m}

```

zoomed $K_{\alpha,\beta}$ spectra of a NaCl-crystal



```
[357]: M=58.44
rho=2.164/(100**3)
N_A=4*M/(a**3*rho)/(1e-2)**12*1e-23
Delta_N_A=N_A**3*sqrt(Delta_a**2/a**2)
N_A,Delta_N_A,_,latexNA=round_to_sigs(N_A,Delta_N_A,r'\1e23\per\mole')
print(latexNA)

M=25.94
rho=2.635/(100**3)
a=201.4*2
N_A=4*M/(a**3*rho)/(1e-2)**12*1e-23
print(N_A)
```

$\backslash\text{qty}\{6.1(0.9)\}\{\backslash\text{1e23}\backslash\text{per}\backslash\text{mole}\}$
6.025331667684615

```
[358]: compare_table_array(r'h_\theta',r'h_\text{Lit}',r'\joule\s',[6.5,6.3],[1.5,0.
↪6],[6.62607015,6.62607015],[0,0])
K_alpha,Delta_K_alpha,_,_=round_to_sigs(np.mean([71.0,71.1]),np.sqrt(np.mean([1.
↪7,1.0])**2+np.std([71.0,71.1])**2),r'')
K_beta,Delta_K_beta,_,_=round_to_sigs(np.mean([63.0,63.2]),np.sqrt(np.mean([1.
↪7,1.0])**2+np.std([63.0,63.3])**2),r'')
compare_table_array(r'K^\text{exp}_\alpha(\text{Mo})',r'K^\text{Lit}_\alpha(\text{Mo})',r'\pico\m',[K_alpha,
↪1,63.1],[0,0])
compare_table(r'N_A^\text{exp}',r'N_A^\text{lit}',r'\per\mole',6.1,0.9,6.
↪02214076,0)
```

```
\begin{table}[htbp]
\centering
\caption{Vergleich von  $h_\theta$  und  $h_\text{Lit}$ }
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
\toprule
Messreihe &  $h_\theta$  [\unit{joule\s}] &  $h_\text{Lit}$  [\unit{joule\s}] & & \\
\text{abs. Abw.} [\unit{joule\s}] & & \text{proz. Abw.} [\unit{percent}] & & S [\sigma] \\
\\ \midrule
& \num{6.5(1.5)} & 6.62607015 & \num{0.1(1.5)} & \num{2(24)} & & \\
\num{0.09} & & & & & & \\
& \num{6.3(0.6)} & 6.62607015 & \num{0.3(0.6)} & \num{5(10)} & \num{0.6} & \\
\\
\bottomrule
\end{tabularx}
\label{table:Vergleich_h_\theta}
\end{table}

\begin{table}[htbp]
\centering
\caption{Vergleich von  $K^\text{exp}_\alpha(\text{Mo})$  und  $K^\text{Lit}_\alpha(\text{Mo})$ }
\begin{tabularx}{0.8\textwidth}{ZZZZZ}
\toprule
Messreihe &  $K^\text{exp}_\alpha(\text{Mo})$  [\unit{pico\m}] & & & \\
 $K^\text{Lit}_\alpha(\text{Mo})$  [\unit{pico\m}] & & \text{abs. Abw.} [\unit{pico\m}] & & \\
\text{proz. Abw.} [\unit{percent}] & & S [\sigma] & & \\
& \num{71.1(1.4)} & 71.1 & 0 & 0 & \num{0} & \\
& \num{63.1(1.4)} & 63.1 & 0 & 0 & \num{0} & \\
\\
\bottomrule
\end{tabularx}
\label{table:Vergleich_K^\text{exp}_\alpha(\text{Mo})}
\end{table}
```

```
\begin{table}[htbp]
\centering
\caption{Vergleich von  $N_A^\text{exp}$  und  $N_A^\text{lit}$ }
\begin{tabularx}{\textwidth}{ZZZZx}
\toprule
```

```

N_A^{\text{exp}}[\text{\unit{per}\mole}]\&N_A^{\text{lit}}[\text{\unit{per}\mole}]\&\text{abs. Abw.}[\text{\unit{per}\mole}]\&\text{proz. Abw.}[\text{\unit{percent}}]\&S[\sigma]\midrule
\text{\num{6.1(0.9)}\&6.02214076\&\num{0.1(0.9)}\&\num{1(15)}\&\num{0.09}}\\
\bottomrule
\end{tabularx}
\label{table:Vergleich_N_A^{\text{exp}}}
\end{table}

```

[359]: `gff("4*M/(rho*d**3)", "d")`

Funktion:

$$\frac{4M}{d^3\rho}$$

`\frac{4 M}{d^{\{3\}} \rho}`

Absoluter Fehler:

$$12\sqrt{\frac{\Delta_d^2 M^2}{d^8 \rho^2}}$$

`12 \sqrt{\frac{\Delta_{d}^{\{2\}} M^{\{2\}}}{d^{\{8\}} \rho^{\{2\}}}}`

Relativer Fehler:

$$3\sqrt{\frac{\Delta_d^2}{d^2}}$$

`3 \sqrt{\frac{\Delta_{d}^{\{2\}}}{d^{\{2\}}}}`

[359]: `(4*M/(d**3*rho),
12*sqrt(Delta_d**2*M**2/(d**8*rho**2)),
3*sqrt(Delta_d**2/d**2),
[(d, Delta_d)])`